# MODUL 306

## Dokumentation Tic Tac Toe vom Projektteam «Planwerk»

Abbildung 1 Projektbild

**Verfasser:** Ahmad Jafari, Tobias Zimmermann, Marc Greub und Saathujan Selvasingham (Projektteam Planwerk)

| | |
|---|---|
| **Klasse:** | **BIS25A** |
| **Modul:** | **306** |
| **Lehrperson:** | **Michal Durik** |

# Inhaltsverzeichnis Planwerk

**1. Arbeitsjournal** ..... 3
**Dokumentation Phase 1 (Fertig)** ..... 4
**2.0 Phase 2. Konzept Tic Tac Toe Applikation** ..... 5
**2.1 Anforderungen & Architektur:** ..... 6
2.1.1. Lastenheft / Systemanforderungen ..... 6
2.1.2. Systemarchitektur ..... 7
2.1.4. Detailstudie ..... 8
**2.2 Konzepte für die spätere Umsetzung:** ..... 8
2.2.1 Betriebskonzept ..... 8
2.2.2. Geschäftsorganisationskonzept ..... 8
2.2.3. Einführungskonzept ..... 9
2.2.4. Testkonzept ..... 9
2.2.5. Integrationskonzept ..... 9
2.2.6. Migrationskonzept ..... 9
**2.3 Evaluation:** ..... 10
2.3.1. Kriterienkatalog ..... 10
2.3.2. Evaluationsbericht ..... 11
**3. Phase: Realisierung** ..... 13
**4. Phase: Einführung** ..... 15
**5. Abbildungsverzeichnis** ..... 16
**6. Quellenverzeichnis** ..... 16

# 1. Arbeitsjournal

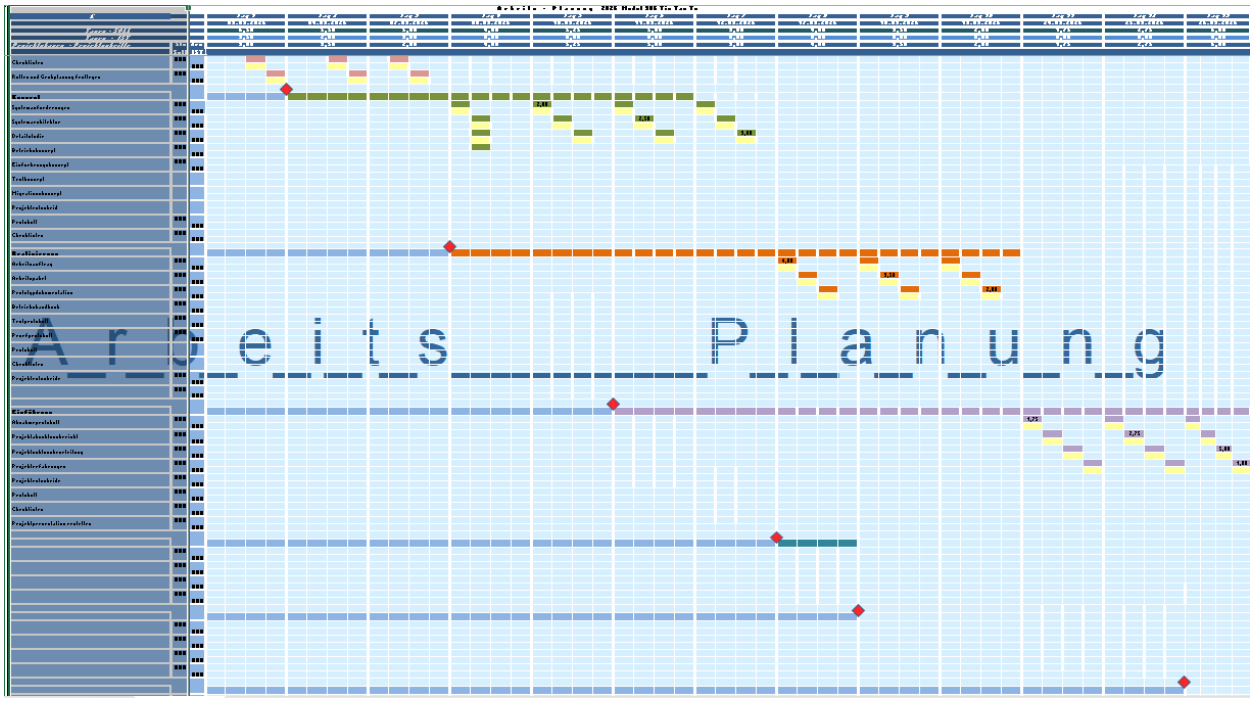


Abbildung 2 Arbeitsjournal

Arbeitsplan 2.xlsx

# Dokumentation Phase 1 (Fertig) nur noch einfügen von Saathu

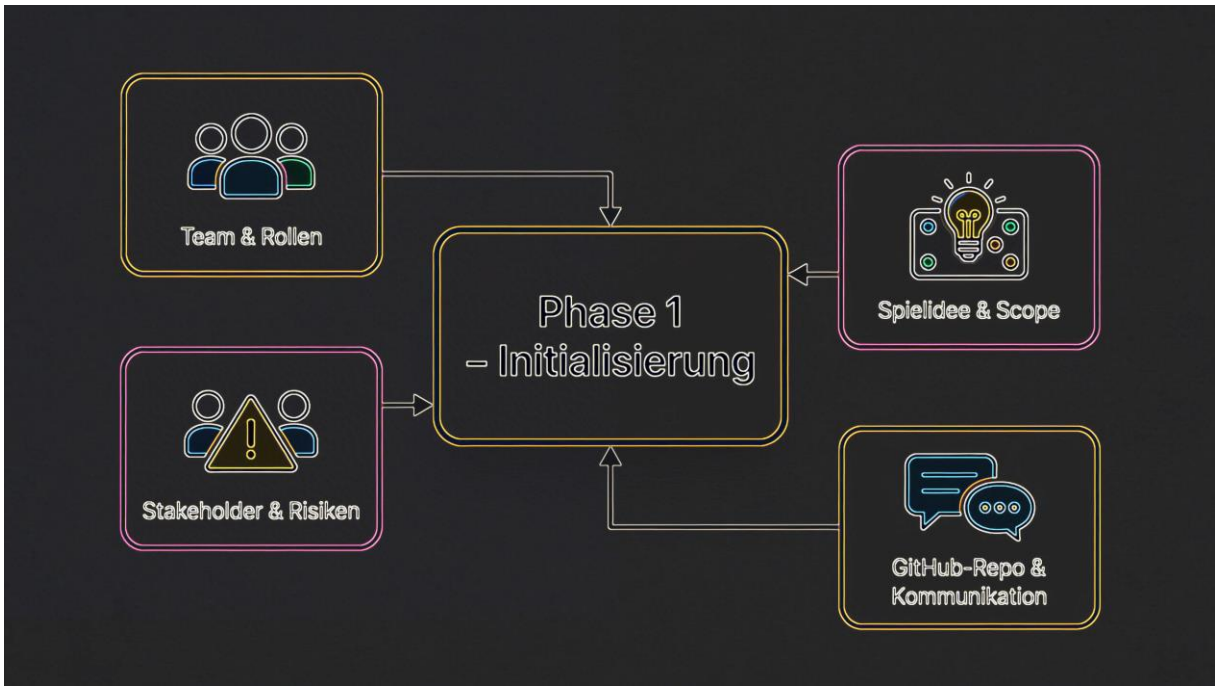


Abbildung 3 Phase 1

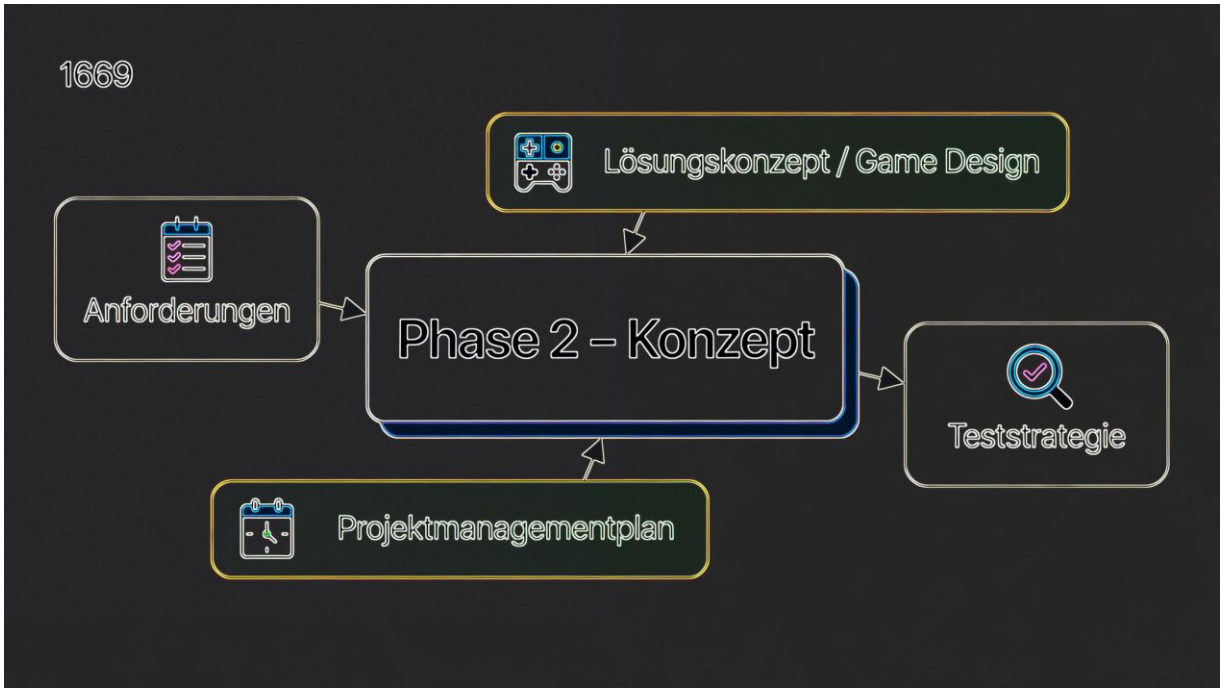


Abbildung 4 Phase 2

# 2.0 Phase 2. Konzept Tic Tac Toe  Applikation

## Einleitung

Im Rahmen dieses Projekts wurde eine webbasierte TicTacToe-Applikation konzipiert und umgesetzt, die registrierten Benutzern ermöglicht, in Echtzeit gegeneinander zu spielen. Die Anwendung umfasst eine Benutzerregistrierung, eine Authentifizierung über E-Mail sowie ein interaktives Spielsystem mit Spieler-Matching.

Zu Beginn erstellt der Benutzer über eine Registrierungsseite ein persönliches Konto. Dabei werden Benutzername, E-Mail-Adresse, Passwort sowie optional ein Profilbild erfasst und im System gespeichert. Nach erfolgreicher Registrierung erhält der Benutzer eine E-Mail mit einem sechsstelligen Verifizierungscode. Dieser Code dient als temporäre Authentifizierung für den Login-Prozess.

Für die Anmeldung gibt der Benutzer seinen Benutzernamen sowie den erhaltenen sechsstelligen Code auf der Login-Seite ein. Nach erfolgreicher Authentifizierung erhält der Benutzer Zugriff auf die TicTacToe-Spieloberfläche.

Das System versucht anschließend automatisch, zwei registrierte Benutzer miteinander zu verbinden. Wenn sich innerhalb einer definierten Wartezeit von zehn Sekunden ein zweiter Spieler anmeldet, wird eine gemeinsame Spielsitzung erstellt. Falls innerhalb dieser Zeit kein weiterer Spieler verfügbar ist, wird automatisch eine integrierte KI als Gegenspieler eingesetzt.

Während des Spiels zeigt die Benutzeroberfläche klar an, welcher Spieler aktuell am Zug ist. Der jeweils andere Spieler muss warten, bis der aktive Spieler ein Feld auf dem Spielfeld auswählt. Nach jeder Aktion wechselt der Zug automatisch zum nächsten Spieler. Das Spiel wird fortgesetzt, bis ein Spieler gewinnt oder das Spiel unentschieden endet.

Nach Abschluss einer Partie wird das Spielergebnis unterhalb des Spielfelds als Match-Historie angezeigt. Zusätzlich wird jeder Sieg in der persönlichen Statistik des jeweiligen Benutzers gespeichert und im Profilbereich dargestellt.

Die Spieloberfläche enthält außerdem das Profilbild, den Benutzernamen sowie die aktuelle Anzahl der gewonnenen Spiele. Nach Beendigung einer Runde können die Spieler über eine „Next Round“-Funktion eine neue Partie starten und das Spiel fortsetzen.

## 2.1 Anforderungen & Architektur:

### 2.1.1. Lastenheft / Systemanforderungen

Die TicTacToe-Applikation soll eine benutzerfreundliche und zuverlässige Plattform bereitstellen, auf der Benutzer online gegeneinander spielen können. Das System muss eine sichere Benutzerverwaltung sowie eine stabile Spielumgebung gewährleisten.

**Funktionale Anforderungen**

Das System muss folgende Funktionen unterstützen:

- Benutzer müssen ein Konto über eine Registrierungsseite erstellen können.
- Benutzer müssen einen Benutzernamen, eine E-Mail-Adresse und ein Passwort eingeben können.
- Benutzer sollen optional ein Profilbild hochladen können.
- Nach der Registrierung muss ein sechsstelliger Verifizierungscode per E-Mail versendet werden.
- Benutzer müssen sich mit ihrem Benutzernamen und dem Verifizierungscode anmelden können.
- Das System soll automatisch zwei Spieler miteinander verbinden.
- Falls kein zweiter Spieler verfügbar ist, soll eine KI als Gegenspieler eingesetzt werden.
- Das Spielsystem muss erkennen, wenn ein Spieler gewonnen hat oder ein Unentschieden entsteht.
- Die Spielergebnisse sollen gespeichert und als Match-Historie angezeigt werden.
- Benutzer sollen jederzeit eine neue Spielrunde starten können.

**Nicht-funktionale Anforderungen**

Neben den funktionalen Anforderungen muss das System folgende Qualitätsanforderungen erfüllen:

- Die Benutzeroberfläche soll einfach und übersichtlich gestaltet sein.
- Das System soll schnell auf Benutzeraktionen reagieren.
- Die Anwendung soll über einen Webbrowser zugänglich sein.
- Benutzerinformationen sollen sicher gespeichert werden.
- Das System soll stabil und zuverlässig funktionieren.

### 2.1.2. Systemarchitektur

Die TicTacToe-Applikation soll eine benutzerfreundliche und zuverlässige Plattform bereitstellen, auf der Benutzer online gegeneinander spielen können. Das System muss eine sichere Benutzerverwaltung sowie eine stabile Spielumgebung gewährleisten.

**Frontend**

Das Frontend stellt die Benutzeroberfläche der Anwendung bereit und wird im Webbrowser ausgeführt. Es wurde mit HTML, CSS und JavaScript umgesetzt. Das Frontend ermöglicht den Benutzern die Registrierung, den Login sowie die Interaktion mit dem TicTacToe-Spielfeld.

**Backend**

Das Backend wurde mit Node.js entwickelt und verwendet das Framework Express.js zur Verarbeitung von Serveranfragen. Der Server verwaltet die Benutzerkonten, steuert den Spielablauf und verarbeitet die Kommunikation zwischen den Spielern.

**Echtzeitkommunikation**

Für die Echtzeitkommunikation zwischen den Spielern wird die Bibliothek Socket.io verwendet. Diese Technologie ermöglicht es, Spielzüge sofort zwischen den beiden Spielern zu übertragen, ohne dass die Seite neu geladen werden muss.

**Datenbank**

Zur Speicherung der Benutzerdaten und Spielergebnisse wird eine MongoDB-Datenbank verwendet. Die Datenbank speichert Informationen wie Benutzerkonten, Profilbilder, Spielresultate sowie die Anzahl der gewonnenen Spiele.

### 2.1.3. Verwendete Technologie

Für die Entwicklung der TicTacToe-Applikation wurden verschiedene Technologien eingesetzt, die eine stabile und interaktive Webanwendung ermöglichen.
Das Frontend wurde mit HTML, CSS und JavaScript entwickelt, um eine benutzerfreundliche Oberfläche zu erstellen.
Das Backend basiert auf Node.js und verwendet das Framework Express.js zur Verwaltung der Serverlogik.
Für zusätzliche Funktionen wurden folgende Technologien eingesetzt:

- Socket.io für die Echtzeitkommunikation zwischen Spielern
- Nodemailer für den Versand des Verifizierungscodes per E-Mail
- Multer für das Hochladen von Profilbildern
- Bcrypt zur sicheren Verschlüsselung von Benutzerpasswörtern
- Dotenv zur Verwaltung von Konfigurationsdaten
- Mongoose zur Verbindung und Strukturierung der MongoDB-Datenbank

Diese Technologien ermöglichen eine sichere, stabile und leistungsfähige Umsetzung der Anwendung.

### 2.1.4. Detailstudie

Im Rahmen der Detailstudie wurde untersucht, welche technische Lösung für die Umsetzung der TicTacToe-Applikation am besten geeignet ist.
Eine webbasierte Lösung wurde gewählt, da sie den Vorteil bietet, dass Benutzer ohne Installation direkt über einen Webbrowser auf die Anwendung zugreifen können. Dadurch wird die Nutzung vereinfacht und die Anwendung kann auf verschiedenen Geräten verwendet werden.
Die Verwendung von Node.js ermöglicht eine effiziente Verarbeitung von Serveranfragen und eignet sich besonders gut für Echtzeitanwendungen. Durch die Integration von Socket.io können Spielzüge sofort zwischen den Spielern übertragen werden, wodurch ein flüssiger Spielablauf gewährleistet wird.
MongoDB wurde als Datenbank gewählt, da sie eine flexible Speicherung von Benutzerdaten und Spielergebnissen ermöglicht und sich gut in Node.js-Anwendungen integrieren lässt.
Die gewählte Systemarchitektur ermöglicht außerdem zukünftige Erweiterungen der Anwendung, beispielsweise Ranglisten, zusätzliche Spielmodi oder erweiterte Spielerstatistiken.

## 2.2 Konzepte für die spätere Umsetzung:

### 2.2.1 Betriebskonzept

Um die TicTacToe-Applikation erfolgreich in Betrieb zu nehmen, werden im Rahmen der HERMES-Phase 2 folgende Konzepte erstellt oder vorbereitet. Diese Konzepte dienen der strukturierten Umsetzung, dem Betrieb und der Einführung der Anwendung:

**Betriebskonzept**
Das Betriebskonzept beschreibt die **technische und organisatorische Infrastruktur**, auf der die Applikation läuft. Dazu gehören:

- Hosting der Webanwendung auf einem Server
- Verwaltung der Node.js- und Express-Instanzen
- Anbindung der MongoDB-Datenbank
- Sicherstellung von Backups und Wartung
- Monitoring der Serverleistung und Fehlerprotokollierung

### 2.2.2. Geschäftsorganisationskonzept

Dieses Konzept definiert **Rollen und Verantwortlichkeiten**, die für den Betrieb der Applikation notwendig sind:

- Verwaltung der Benutzerkonten
- Support bei Problemen oder Fehlermeldungen
- Überwachung der Systemleistung

### 2.2.3. Einführungskonzept

Das Einführungskonzept beschreibt, **wie die Benutzer die Anwendung nutzen** und wie die Einführung geplant ist:

- Registrierung und Login der Benutzer
- Bereitstellung von Tutorials oder Hilfetexten
- Schrittweises Hochfahren des Systems bei mehreren Benutzern

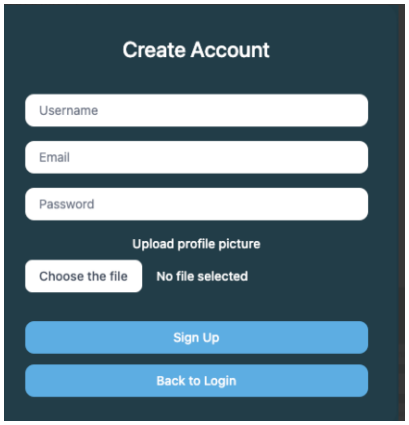


Abbildung 5 Account erstellen

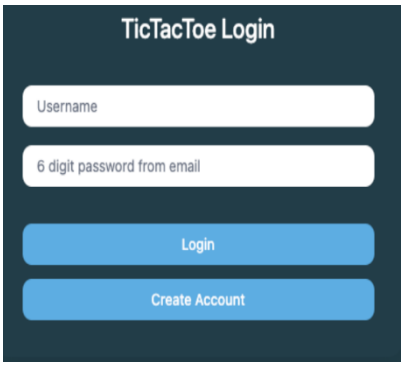


Abbildung 6 Login

### 2.2.4. Testkonzept

Das Testkonzept legt fest, **wie die Funktionsfähigkeit und Stabilität geprüft werden**:

- Tests der Registrierung und Anmeldung
- Tests des Spieler-Matching
- Überprüfung der Echtzeit-Kommunikation
- Validierung der Match-History und Spielerstatistiken
- Sicherstellung der korrekten Funktion der KI

### 2.2.5. Integrationskonzept

Das Integrationskonzept beschreibt, **wie die einzelnen Komponenten miteinander verbunden werden**:

- Frontend ↔ Backend über HTTP und Socket.io
- Backend ↔ Datenbank über Mongoose
- Backend ↔ E-Mail-Service über Nodemailer

### 2.2.6. Migrationskonzept

| ID | Kriterium | GW (1-5) | Gewählte Technologie | Alternative 1 | Alternative 2 | Beschreibung |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| K-01 | Backend | 5 | Node.js | Deno | Python (Flask) | |
| K-02 | Echtzeit-Kommunikation | 5 | Socket.io | WebSockets | Server-Sent-Event | |
| K-03 | Datenbank | 3 | MongoDB | PostgresSQL | Firebase | |
| K-04 | Frontend | 2 | HTML /CSS | - | - | |
| K-05 | Build-Tool | 2 | dotenv | Cross-env | - | |
| K-06 | Version Verwaltung | 4 | Github | Gitlab | Bitbucket | |

Das Migrationskonzept legt fest, **wie bestehende Daten oder zukünftige Updates eingespielt werden**:

- Sicherung der bestehenden Benutzerdaten
- Migration neuer Spielfunktionen ohne Datenverlust
- Aktualisierung der Datenbankstrukturen bei Bedarf

## 2.3 Evaluation:

### 2.3.1. Kriterienkatalog

Kriterienkatalog

| ID | Kriterium | Gewicht | Node.js | Deno | Python | Socket.io | WebSocket | MongoDB | PostgreSQ | Auswahl | Begründung |
|---|---|---|---|---|---|---|---|---|---|---|---|
| K01 | Echtzeitfähigkeit | 5 | ++ | + | O | ++ | + | + | n/a | Node.js + Socket.io | Optimale Performance für bidirektionale Spielzüge |
| K02 | Einarbeitungszeit | 4 | ++ | O | + | ++ | O | ++ | + | Node.js + MongoDB | Bestehende JavaScript-Kenntnisse nutzbar |
| K03 | Community-Größe | 4 | ++ | O | ++ | ++ | + | ++ | ++ | Node.js + MongoDB | Große Community = schnelle Hilfe |
| K04 | Dokumentation | 4 | ++ | O | ++ | ++ | + | ++ | ++ | Node.js + MongoDB | Ausführliche Docs und Tutorials |
| K05 | Flexibilität | 3 | ++ | + | + | ++ | + | ++ | O | MongoDB | JSON-Struktur passt perfekt zu JavaScript |
| K06 | Kosten | 3 | ++ | ++ | ++ | ++ | ++ | ++ | ++ | OSS | Alle Technologien sind Open Source |
| K07 | Skalierbarkeit | 3 | ++ | + | O | ++ | + | ++ | + | Node.js/MongoDB | Gut horizontal skalierbar |
| K08 | Einfachheit | 4 | ++ | O | + | ++ | O | ++ | + | Node.js/Socket.io | Weniger Boilerplate-Code |
| K09 | Sicherheit | 3 | + | ++ | + | ++ | + | + | ++ | Socket.io/PostgreSQL | Eingebaute Sicherheitsfeatures |
| K10 | Wartbarkeit | 3 | ++ | O | + | ++ | + | ++ | + | Node.js/MongoDB | Klare Strukturen und Patterns |

Legende: ++ = sehr gut (2 Pkt.) | + = gut (1 Pkt.) | O = neutral (0 Pkt.) | n/a = nicht anwendbar  Gewicht (1-5)

Abbildung 7 Kriterienkatalog

| Gewichte Auswertung | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Technologie | K01 (5) | K02 (4) | K03 (4) | K04 (4) | K05 (3) | K06 (3) | K07 (3) | K08 (4) | K09 (3) | K10 (3) | GESAMT |
| Node.js | 5x2=10 | 4x2=8 | 4x2=8 | 4x2=8 | 3x2=6 | 3x2=6 | 3x2=6 | 4x2=8 | 3x1=3 | 3x2=6 | 69 |
| Deno | 5x1=5 | 4x0=0 | 4x0=0 | 4x0=0 | 3x1=3 | 0 | 3x1=3 | 4x0=0 | 3x2=6 | 3x0=0 | 23 |
| Python | 5x0=0 | 4x1=4 | 4x2=8 | 4x2=8 | 3x1=3 | 3x2=6 | 3x0=0 | 4x1=4 | 3x1=3 | 3x1=3 | 39 |
| Socket.io | 5x2=10 | 4x2=8 | 4x2=8 | 4x2=8 | 3x2=6 | 3x2=6 | 3x2=6 | 4x2=8 | 3x2=6 | 3x2=6 | 72 |
| WebSockets | 5x1=5 | 4x0=0 | 4x1=4 | 4x1=4 | 3x1=3 | 3x2=6 | 3x1=3 | 4x0=0 | 3x1=3 | 3x1=3 | 31 |
| MongoDB | 5x1=5 | 4x2=8 | 4x2=8 | 4x2=8 | 3x2=6 | 3x2=6 | 3x2=6 | 4x2=8 | 3x1=3 | 3x2=6 | 64 |
| PostgreSQL | n/a | 4x1=4 | 4x2=8 | 4x2=8 | 3x0=0 | 3x2=6 | 3x1=3 | 4x1=4 | 3x2=6 | 3x1=3 | 42 |

Abbildung 8 Gewichte Auswertung

| Entscheidungsmatrix | | | |
|---|---|---|---|
| Rang | Technolog | Gesamtpun | Status |
| 1 | Socket.io | 72 | GEWÄHLT |
| 2 | Node.js | 69 | GEWÄHLT |
| 3 | MongoDB | 64 | GEWÄHLT |
| 4 | PostgreSQL | 42 | Nicht gewählt |
| 5 | Python | 39 | Nicht gewählt |
| 6 | WebSockets | 31 | Nicht gewählt |
| 7 | Deno | 23 | Nicht gewählt |

Abbildung 9 Entscheidungsmatrix

Link: Kriterienkatalog.xlsx

## 2.3.2. Evaluationsbericht

### 1. Bild: Kriterienkatalog

Im ersten Schritt habe ich einen Kriterienkatalog erstellt, um die verschiedenen Technologien fair und nachvollziehbar miteinander vergleichen zu können. Dazu habe ich zehn Kriterien definiert – zum Beispiel Echtzeitfähigkeit, Einarbeitungszeit, Community-Größe, Dokumentation, Flexibilität, Kosten, Skalierbarkeit, Einfachheit, Sicherheit und Wartbarkeit. Jedes Kriterium bekommt eine Gewichtung von 1 bis 5 Punkten, je nachdem, wie wichtig dieser Aspekt für mein Projekt ist; besonders zentrale Themen wie die Echtzeitfähigkeit sind entsprechend hoch gewichtet.

Anschließend bewerte ich jede Technologie (Node.js, Deno, Python, Socket.io, WebSockets, MongoDB und PostgreSQL) pro Kriterium mit ++, + oder 0; „++“ steht für sehr gut, „+“ für gut und „0“ für neutral, „n/a“ bedeutet, dass das Kriterium hier keine Rolle spielt. In der letzten Spalte begründe ich meine Bewertung kurz in Worten, etwa mit vorhandenen Kenntnissen,

Performance, Dokumentationsqualität oder Sicherheitseigenschaften, damit später klar ist, warum ich zu dieser Einschätzung gekommen bin. Auf diese Weise entsteht ein transparenter Überblick darüber, wie gut jede Technologie die für das Projekt wichtigen Anforderungen erfüllt.

**2. Bild: Gewichte-Auswertung**

Im zweiten Schritt überführe ich die eher „gefühlsmässigen“ Bewertungen in Zahlen, damit ich die Technologien objektiver vergleichen kann. Die Symbolbewertungen werden in Punkte umgerechnet und mit der Gewichtung des jeweiligen Kriteriums multipliziert – zum Beispiel ergibt ein Kriterium mit Gewicht 5 und Bewertung „++“ den Teilwert 5×2=10. In der Tabelle „Gewichte-Auswertung“ sehe ich für jede Technologie und jedes Kriterium genau diesen Rechenschritt, zum Beispiel 4×2=8 oder 3×1=3.

Am Ende jeder Zeile wird die Summe aller Teilwerte gebildet – so erhält jede Technologie eine Gesamtpunktzahl über alle Kriterien hinweg. Dadurch wird sichtbar, welche Technologie im Gesamtpaket am besten zu den definierten Anforderungen passt, nicht nur in einem einzelnen Kriterium. Diese Auswertung ist die Grundlage für die eigentliche Entscheidung im nächsten Schritt.

**3. Bild: Entscheidungsmatrix**

Im dritten Schritt fasse ich die Resultate in einer Entscheidungsmatrix zusammen, um eine klare Entscheidung treffen und dokumentieren zu können. In dieser Matrix liste ich alle Technologien mit ihrer Gesamtpunktzahl auf und sortiere sie nach Rang – die höchste Punktzahl erhält Rang 1, die nächsthöchste Rang 2 und so weiter. Zusätzlich bekommt jede Technologie einen Status: „GEWÄHLT“ für diejenigen, die ins Zielsystem übernommen werden, und „Nicht gewählt“ für die übrigen.

In meinem Projekt liegen Socket.io, Node.js und MongoDB vorne und werden deshalb ausgewählt, während PostgreSQL, Python, WebSockets und Deno zwar betrachtet, aber nicht eingesetzt werden. Damit ist die Variantenbewertung im Sinne von HERMES abgeschlossen: Ich habe Alternativen beschrieben, Kriterien definiert und gewichtet, diese systematisch bewertet und die favorisierte Lösungsvariante begründet ausgewählt.

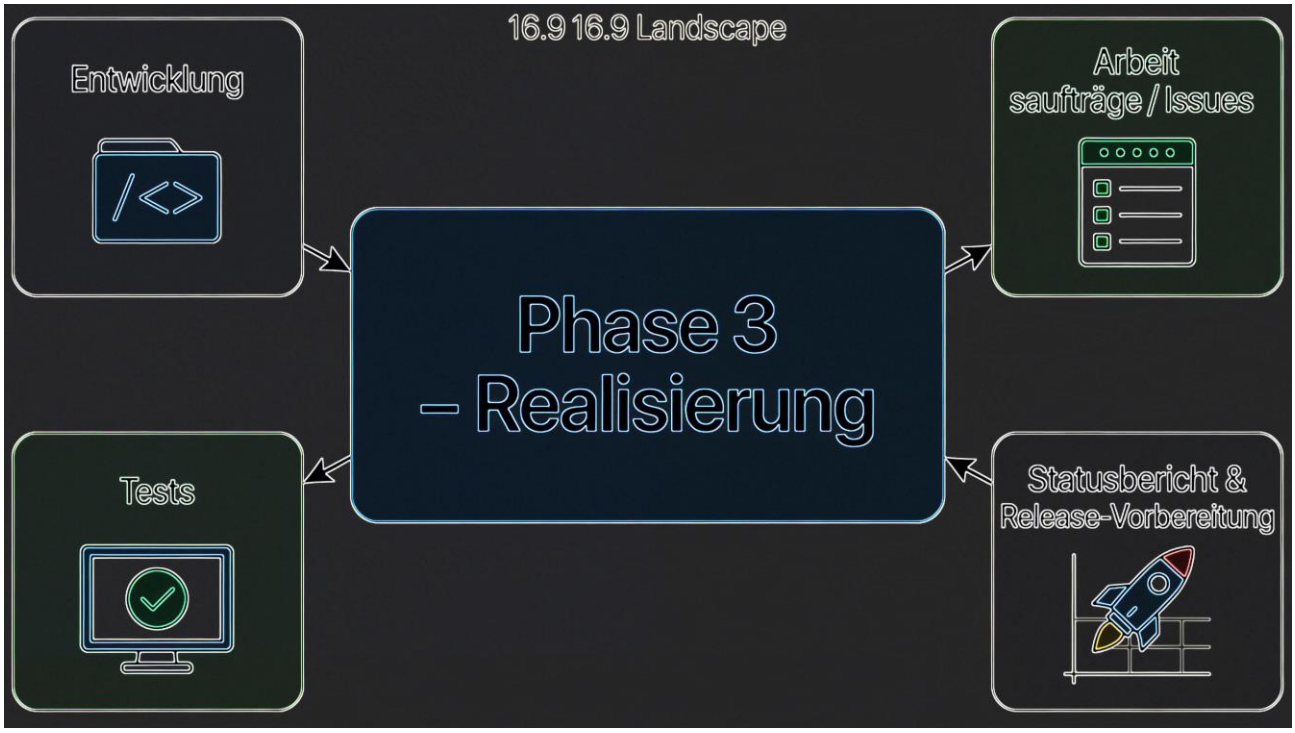


Abbildung 10 Phase 3

# 3. Phase: Realisierung

Die eigentliche Umsetzung des Projekts. Hier werden die Arbeitspakete abgearbeitet, das System entwickelt/gekauft und getestet.

Umsetzung & Steuerung:

- arbeitsauftrag.docx / Vorlage Arbeitspaket.doc (Vergabe von Tasks)

- prototypdokumentation.docx

Dokumentation & Testen:

- betriebshandbuch.docx (wird in der Realisierung verfasst)

- testprotokoll.docx

- pruefprotokoll.xlsx

**Phase 3: Realisierung (Tag 3–4)**
Ziel: Umsetzung gemäss Konzept; Steuerung, Qualität und Änderungen nachvollziehbar.
Aufgaben
1. Entwicklung gemäss Architektur und Anforderungen
2. Arbeitsaufträge über GitHub Issues (inkl. Akzeptanzkriterien)

3. Projektsteuerung:
    - kurzes Daily/Stand-up im Team
    - Statusbericht an Auftraggeber (mind. 1× bis Tag 4)
4. Tests:
    - Testfälle definieren
    - Testdurchführung dokumentieren (Unit/Integration/Gameplay)
5. Benutzerhandbuch (Entwurf) und Release-Vorbereitung

Verpflichtende HERMES-Ergebnisse
- Arbeitsaufträge (über GitHub Issues oder als Dokument)
- Projektstatusbericht (mind. 1×)
- Testkonzept & Testprotokoll
- Benutzerhandbuch (Entwurf)

Meilenstein M3 – Zwischenabnahme Realisierung
- lauffähiger Zwischenstand + Testnachweise
- Statusbericht und offene Risiken/Änderungen transparent

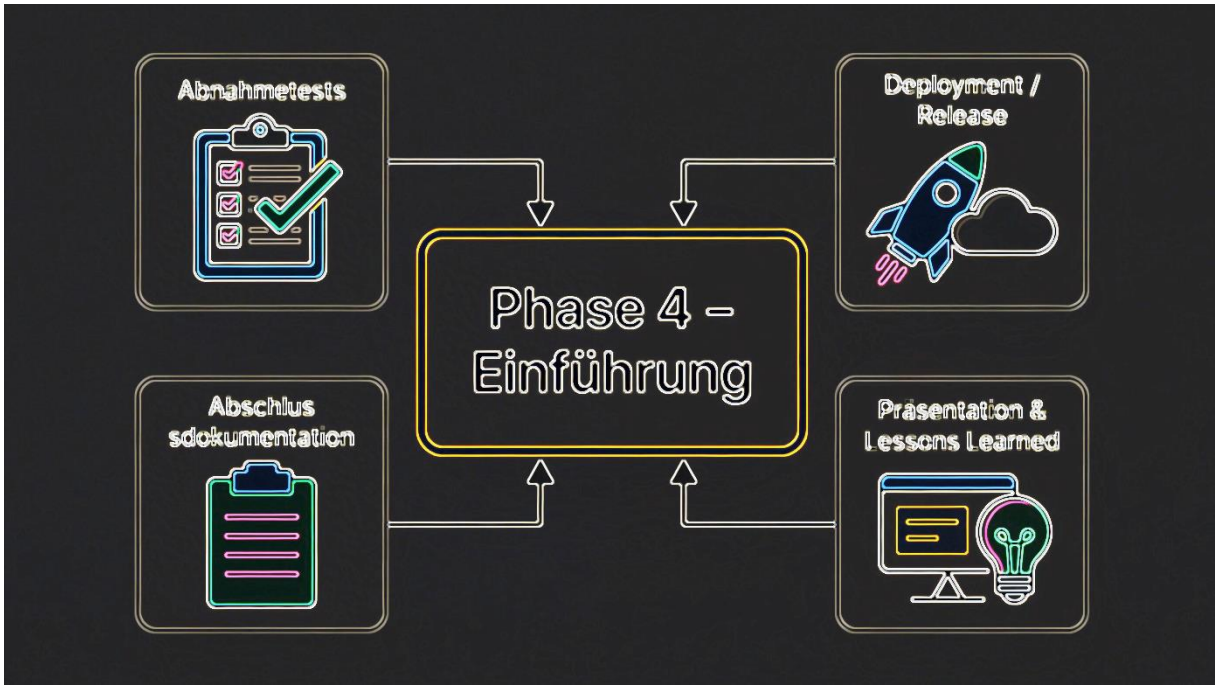


Abbildung 11 Phase 4

# 4. Phase: Einführung

Das System wird in Betrieb genommen, abgenommen und das Projekt formal abgeschlossen.

Abnahme & Übergabe:

- abnahmeprotokoll.docx

Projektabschluss & Reflexion:

- 03_Projektabschlussbericht.docx
- projektschlussbeurteilung.docx
- projekterfahrungen.docx (Lessons Learned)

Phasenübergreifende Dokumente (Führung & Steuerung)

Kommunikation & Meilensteine:

- projektentscheide.docx (Festhalten von Phasenfreigaben)
- protokoll.docx
- checklisten.docx
- projektpraesentation.pptx (Abschlusspräsentation)

# 5. Abbildungsverzeichnis

ABBILDUNG 1 PROJEKTBILD 1
ABBILDUNG 2 ARBEITSJOURNAL 3
ABBILDUNG 3 PHASE 1 4
ABBILDUNG 4 PHASE 2 5
ABBILDUNG 5 ACCOUNT ERSTELLEN 9
ABBILDUNG 6 LOGIN 9
ABBILDUNG 7 KRITERIENKATALOG 10
ABBILDUNG 8 GEWICHTE AUSWERTUNG 11
ABBILDUNG 9 ENTSCHEIDUNGSMATRIX 11
ABBILDUNG 10 PHASE 3 13
ABBILDUNG 11 PHASE 4 15

# 6. Quellenverzeichnis

GitHub Link
Arbeitsplan 2.xlsx
Kriterienkatalog.xlsx